

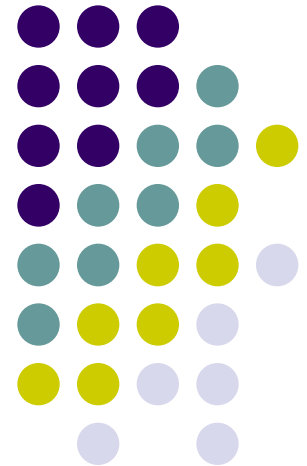
CS4432: Database Systems II

Query Operator & Algebraic Expressions

REFRESH



**YOUR
MEMORY!!!**



Updates

- Quiz 4 is Today
- Project 2 is out
 - Deadline: **Wed Apr 21**
- **No class next Monday (Patriots Day)**
- **Class on Thursday will be by Ben (Chun-Kit) Ngan**

What do we have left

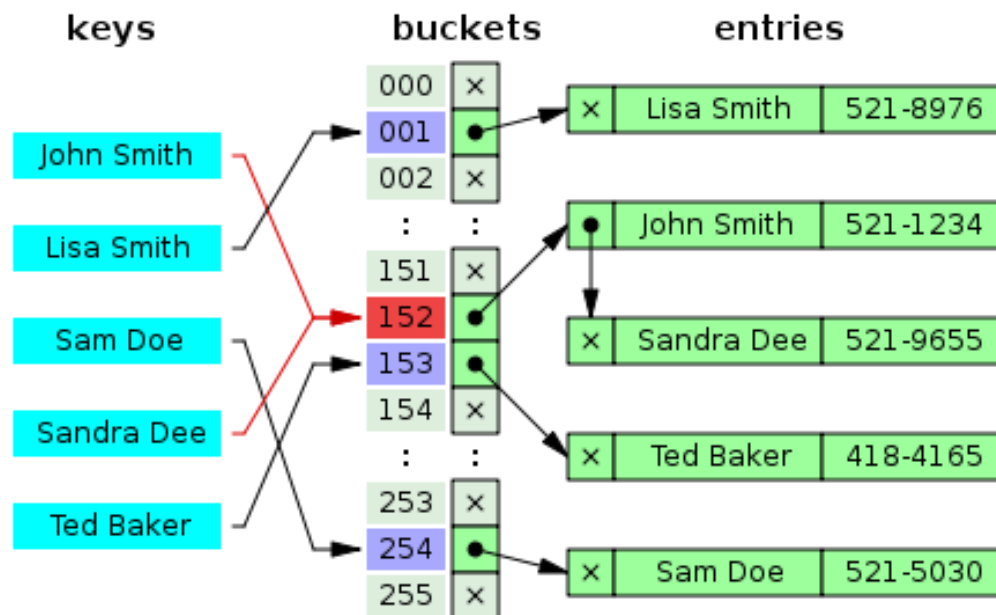
- Quizzes
 - Quiz 4 (today)
 - Recall only 3 best count
- HW + Projects
 - HW 3 and Project 3 will be posted next week
 - You will have ~**10 days to submit** -> please pay attention
- Final exam

What do we have left

- Final Exam:
 - In class, closed book, no communication
 - Everything we learned in class
 - Around 25-30 closed-ended questions (similar to the quizzes)
 - We'll have 75 minutes to complete
- If you have formal accommodation or a problem taking the exam. **Let me know ASAP (now)**

Disk-Based Hash-Based Indexes

- Adaptation of main memory hash tables
- Support *equality* searches
- *No range searches*



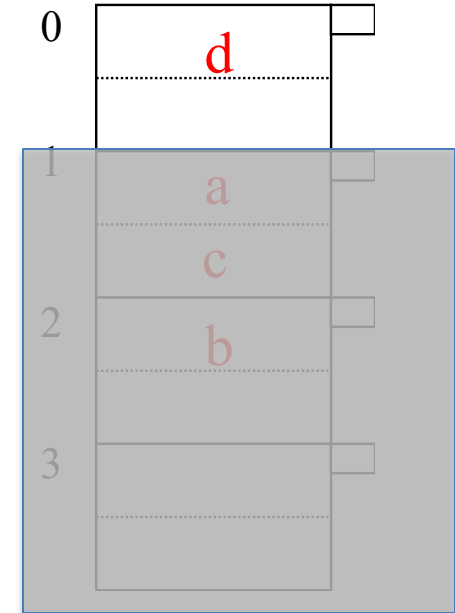
Hash Table: Lookup

Search for key = d



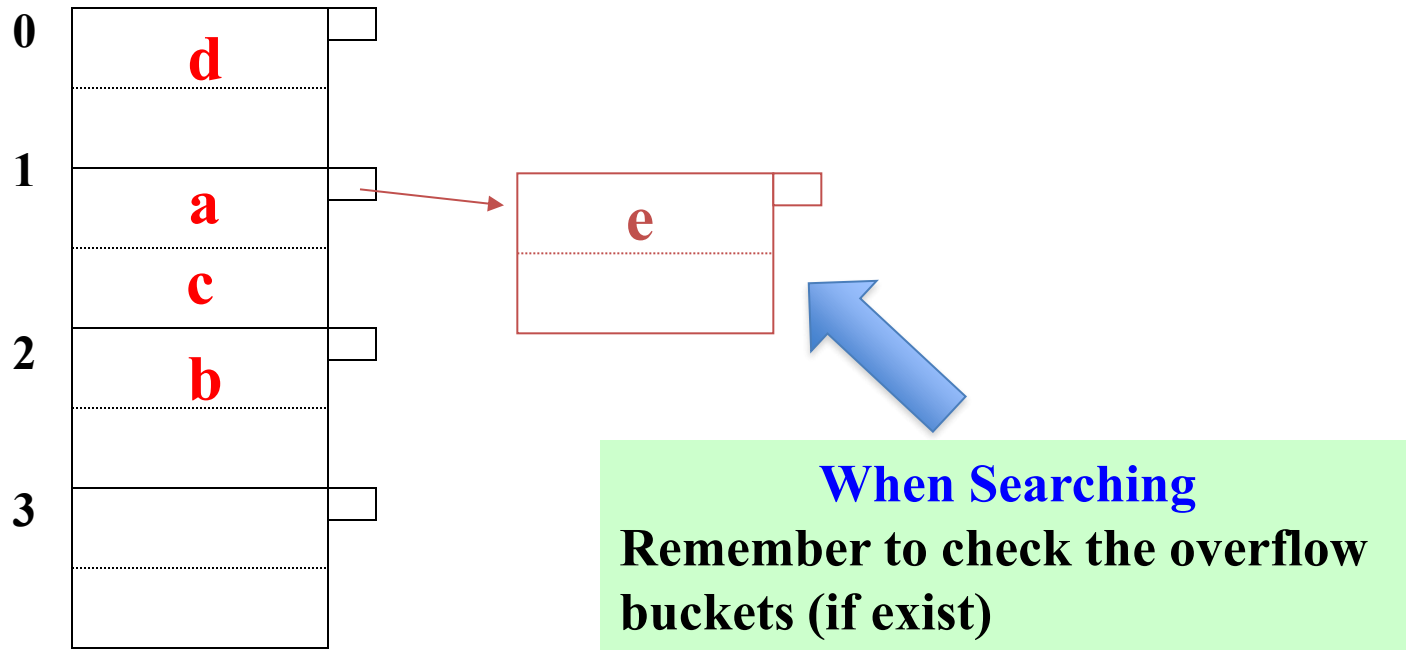
Remember: Only equality search

- 1- Apply the hash function over d $\rightarrow h(d) \bmod N = 0$
- 2- Read the disk page of bucket 0
- 3- Search for key d
 - If keys are sorted, then search using Binary search



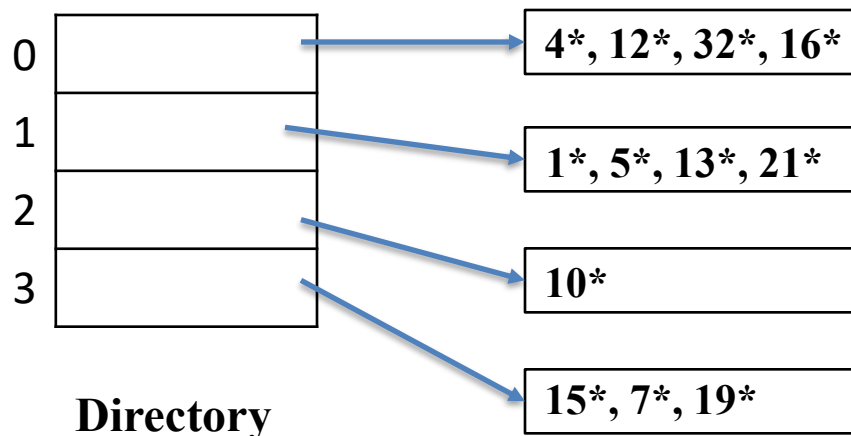
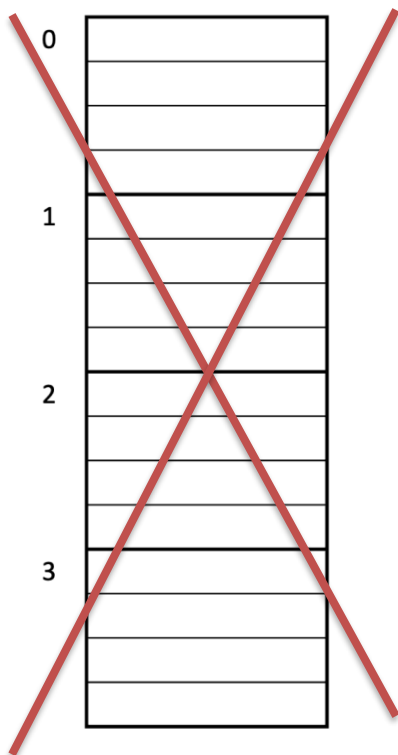
Hash Table: Insertion with Overflow

- Insert key $e \rightarrow h(e) \bmod N = 1$
- Create an **overflow bucket** and insert e
- Overflow bucket is another disk block



Extensible Hash Index

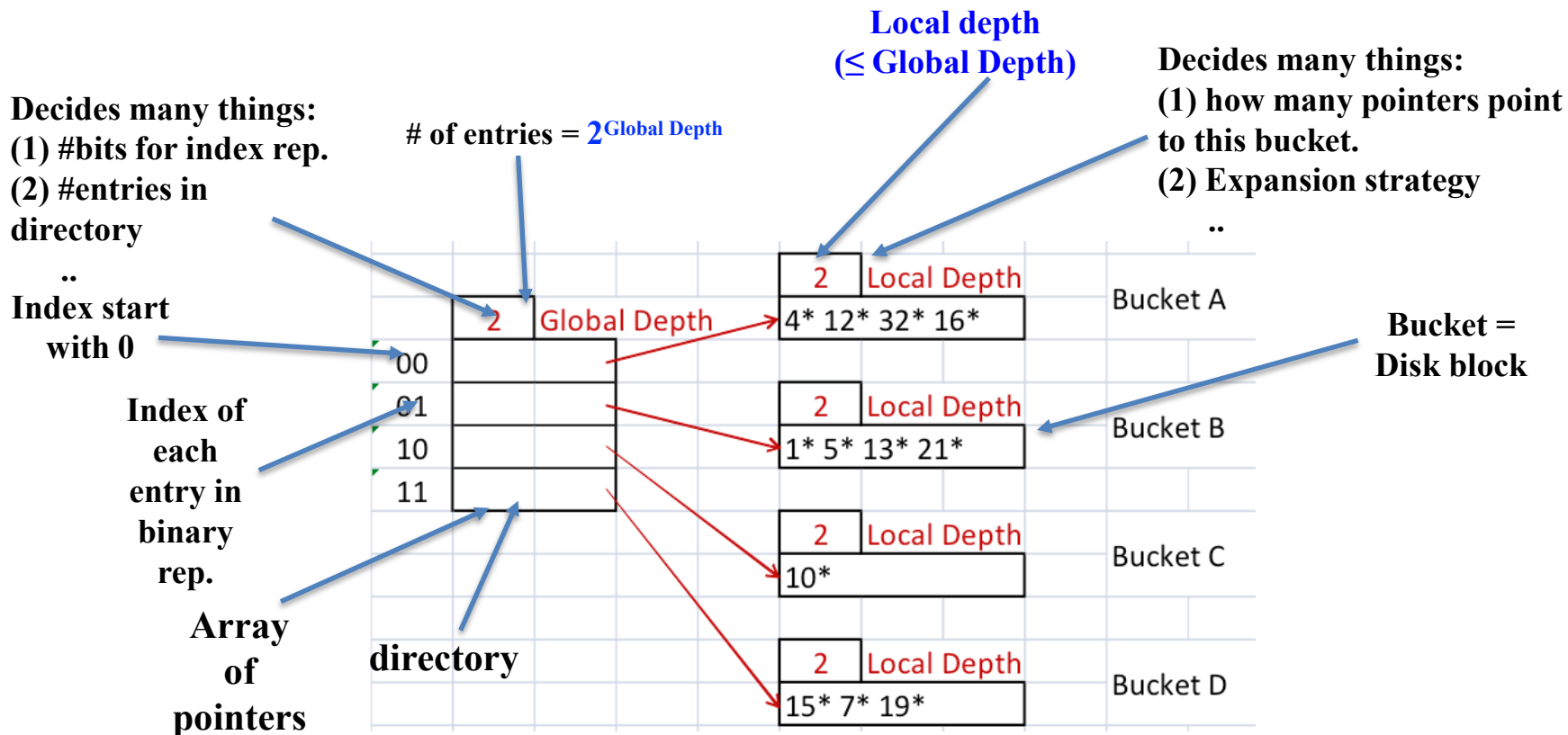
- Start with some # of buckets (e.g., $N=4$ and $h(k) = k$)
- Each bucket can fit fixed # of entries (e.g., $M=4$)
- **Problem:** storing index entries directly on primary bucket eventually requires overflow buckets
- **Solution:** Rather than storing entries directly, use a directory



**What to do when
bucket becomes full?**

Extensible Hash Index

- Start with some # of buckets (e.g., $N=4$)
- Each bucket can fit fixed # of entries (e.g., $M=4$)

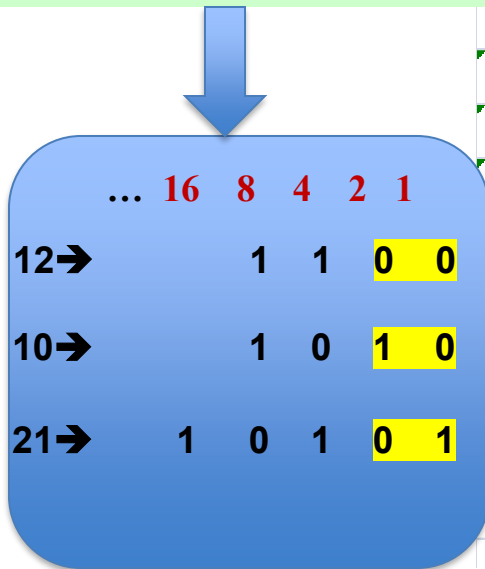


Extensible Hashing: Example

- Directory uses 2 bits (the right-most ones) → 4 entries
- Directory size = 4
- Each bucket holds at most 4 entries

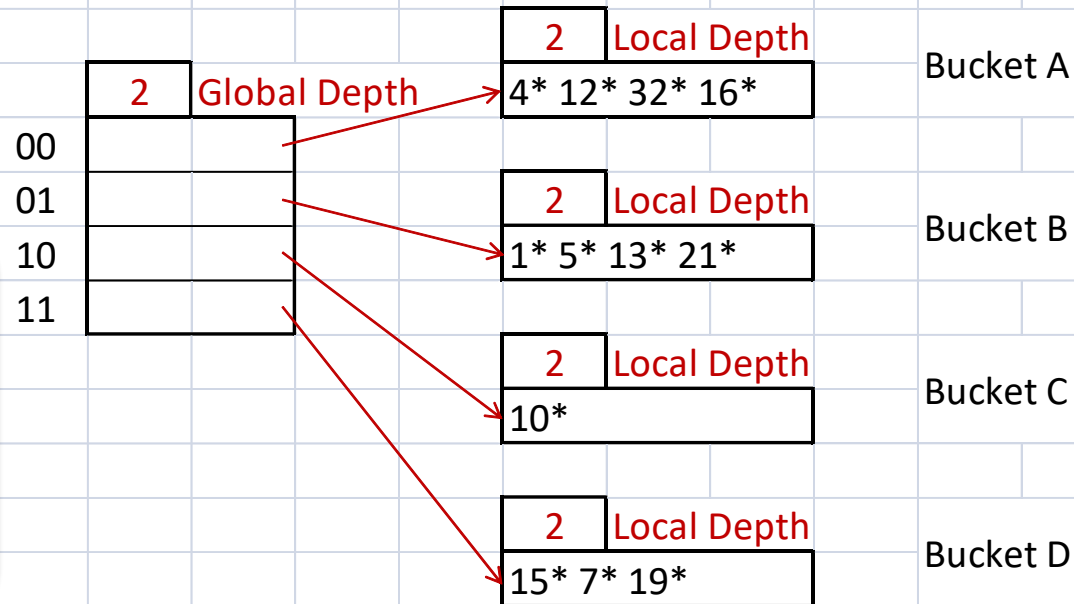
Do it yourself 😊

How did we
insert values
12, 10, 21?



The last two <global-depth> bits determine the bucket

In the beginning...



Inserting Key 20

CASE 1: Expand under (Local Depth = Global Depth)

- 1- Increment global depth
- 2- This means → double the directory size

- 3- For the overflow bucket, divide into two
- 4- Increment their local depth
- 5- Re-distribute the keys *based on the new local depth bits*

- 6- For all other buckets, leave them as is

(local depth < global depth)

- 7- Number of incoming pointers

to each of these bucket is doubled

- For Buckets A & A2 → Keys are distributed based on 3 bits
- For Others → Keys are distributed based on 2 bits

Adding a value with $h(r) = 20$

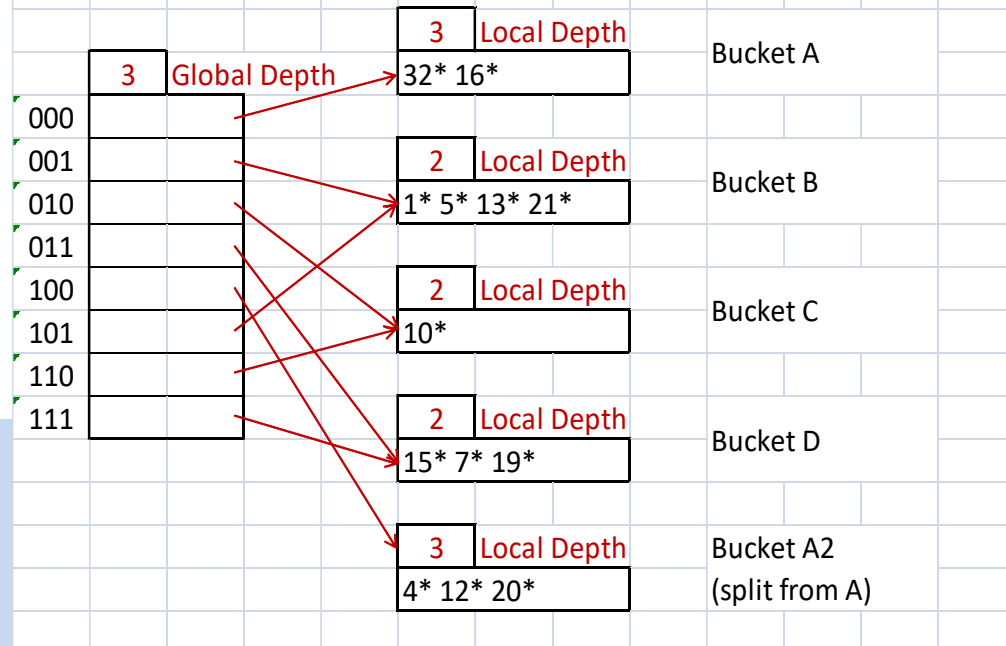
Binary 20 = 10100, maps to Bucket A

Bucket A has no room; what to do?

Split bucket A into two

The Directory needs to be split to accommodate A and A2

Happy ending



Indexing in the AI Era

AI-based Index Structures

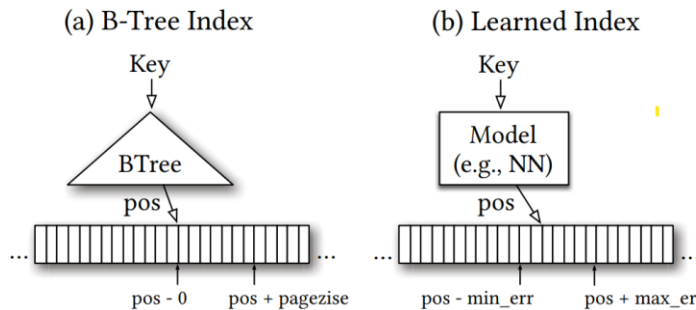


Figure 1: Why B-Trees are models

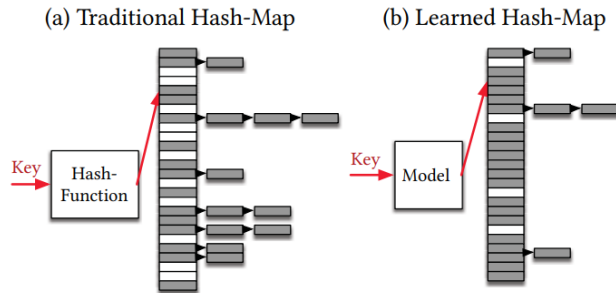


Figure 7: Traditional Hash-map vs Learned Hash-map

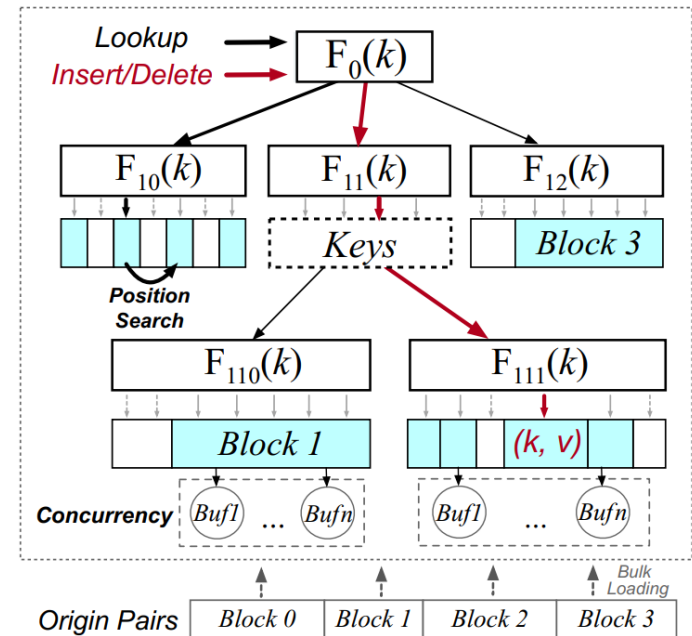


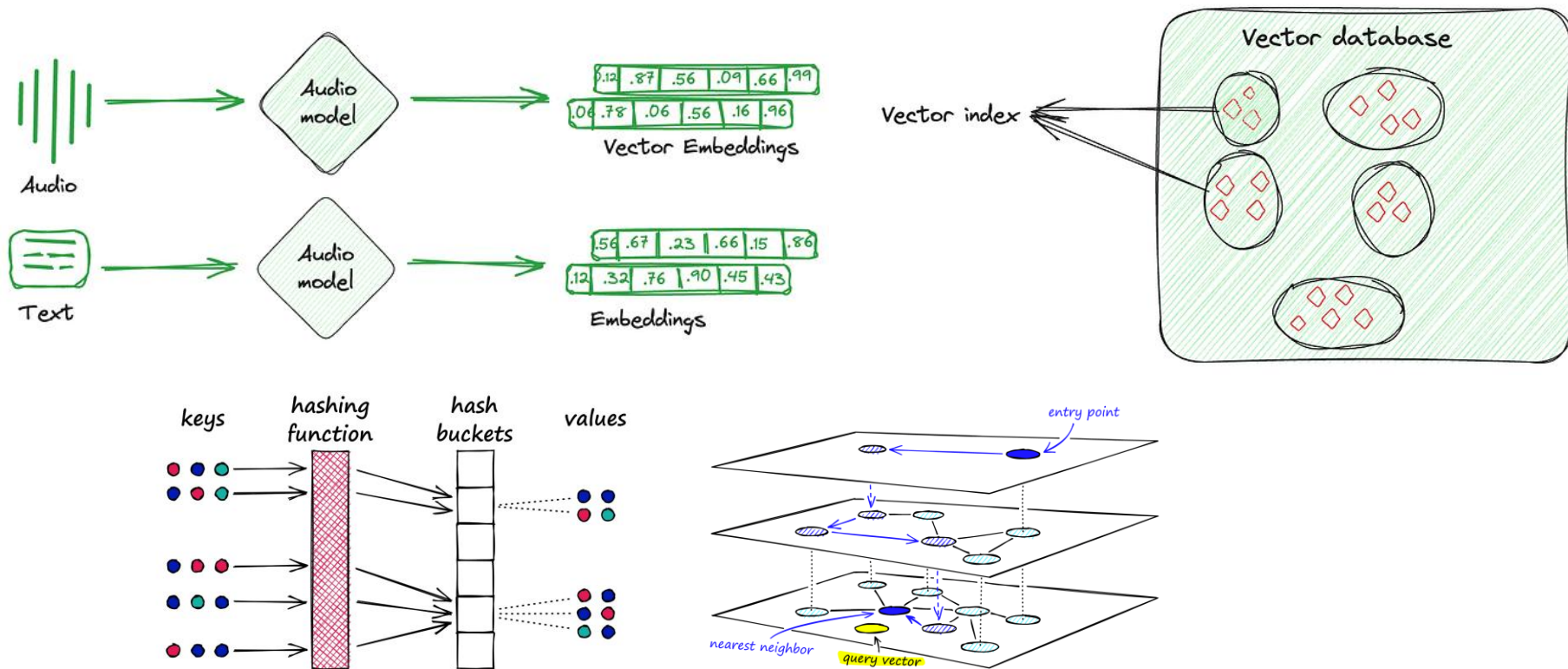
Figure 1: A General Learned Index Structure. Dotted rectangles denote some indexes have no keys in internal nodes.

Sun, Z., Zhou, X., & Li, G. (2023). Learned index: A comprehensive experimental evaluation. *Proceedings of the VLDB Endowment*, 16(8), 1992-2004.

Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Polyzotis, N. (2018, May). The case for learned index structures. In *Proceedings of the 2018 international conference on management of data* (pp. 489-504).

Indexing in the AI Era

Vector DB Systems



<https://www.pinecone.io/learn/series/faiss>

<https://medium.com/@myscale/understanding-vector-indexing-a-comprehensive-guide-d1abe36ccd3c>

Pan, J. J., Wang, J., & Li, G. (2024). Survey of vector database management systems. *The VLDB Journal*, 33(5), 1591-1615.

Week 6

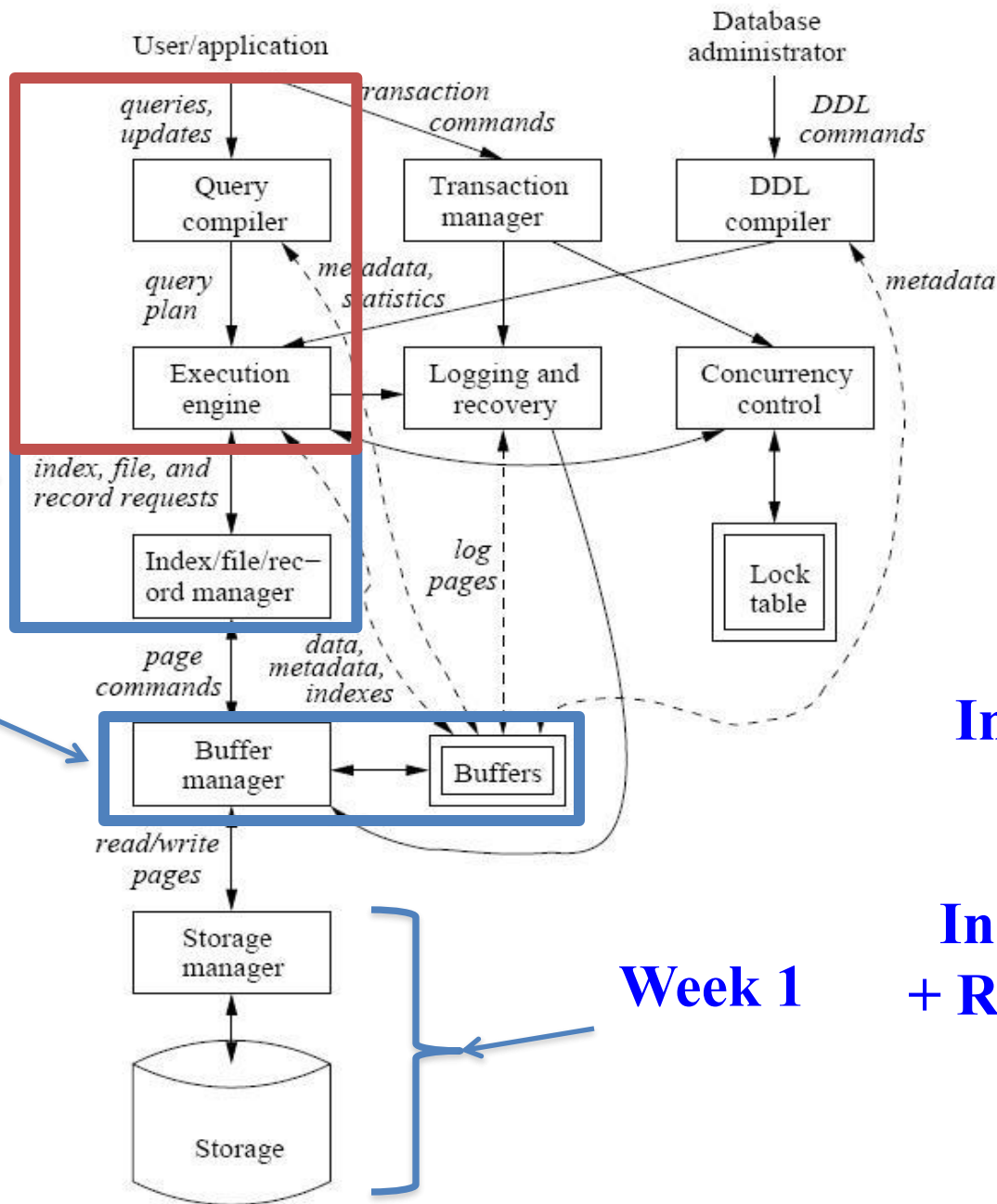
**Week 3:
sorting +**

Week 2

Week 1

**Week 4:
Index via B+Tree**

**Week 5:
Index via Hashing
+ Relational Algebra**





Select B, D

From R, S

Where R.A = "c"

And S.E = 2 And R.C = S.C

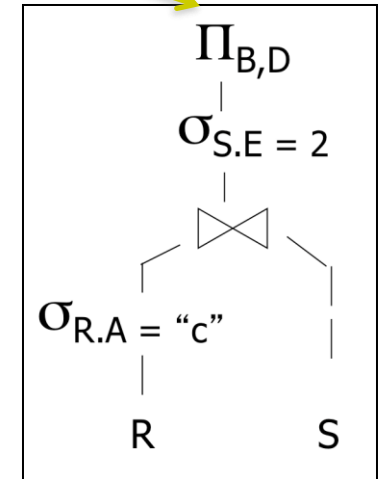
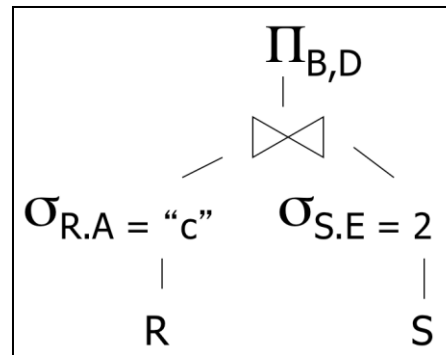
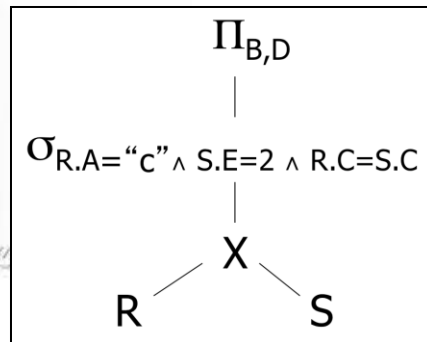
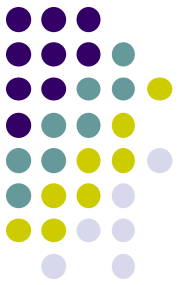


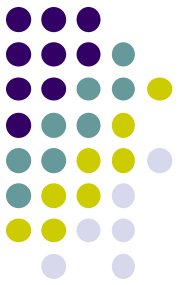


Diagram illustrating nested query rewriting:

- Outer query: σ_C
- Inner query: $\sigma_{R.A="c" \wedge S.E=2 \wedge R.C=S.C}$
- Join operation: X (Join of R and S)
- Projection operation: $\pi_{B,D}$

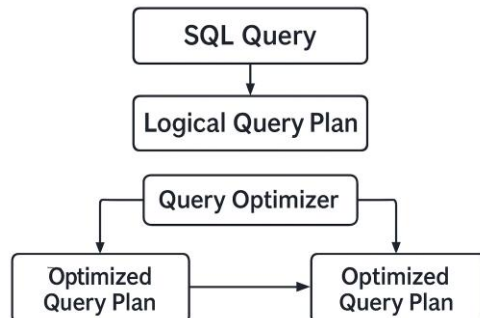


SQL & Query Processing

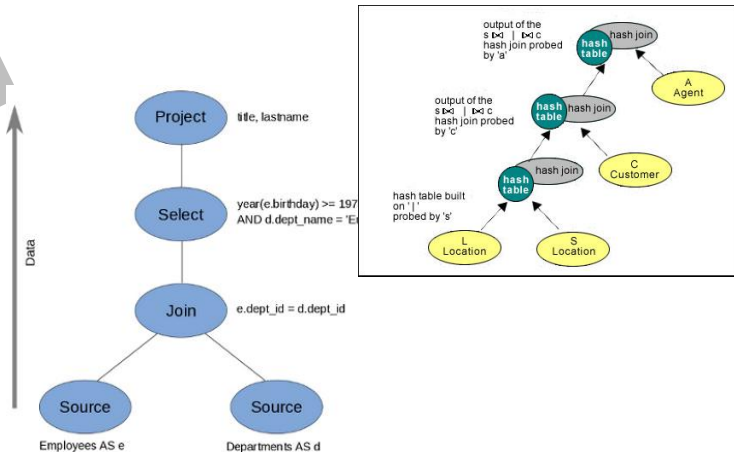


SQL Query

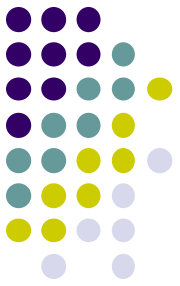
```
SELECT pNumber, count(*) AS CNT
FROM Student
WHERE sNumber > 1
GROUP BY pNumber;
```



Query Plans



Relational Algebra Express Query Plans

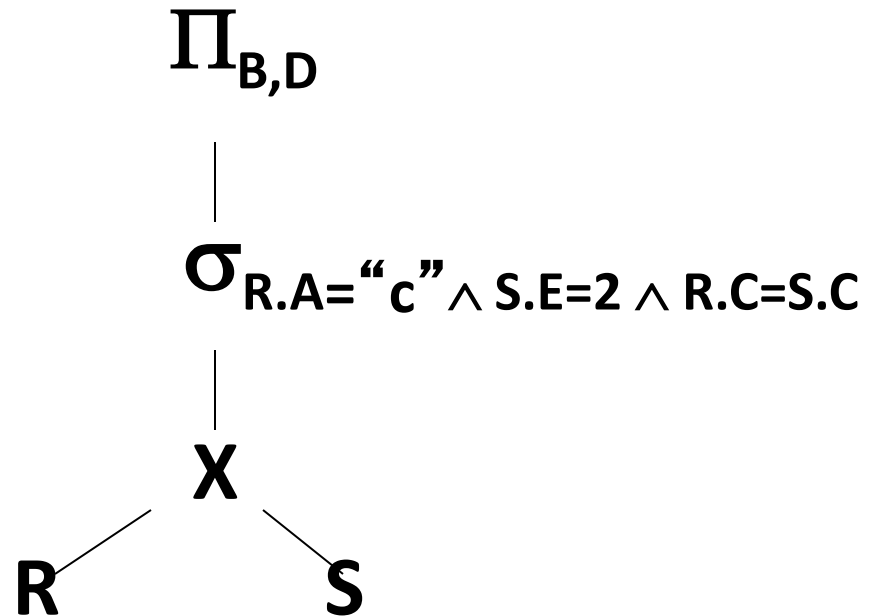


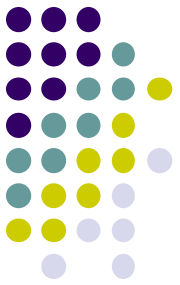
Select B, D

From R, S

Where R.A = "c"

And S.E = 2 **And** R.C=S.C



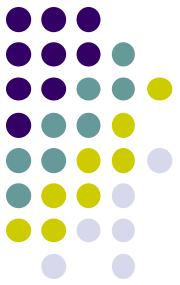


Formal Relational Query Languages

- Relational Algebra: More operational, very useful for representing execution plans.
 - Operators working on relations
- Relational Calculus: Let users describe WHAT they want, rather than HOW to compute it. (Non-operational, declarative.)

$$\{t \mid \exists s \in instructor (t[name] = s[name] \\ \wedge \exists u \in department (u[dept_name] = s[dept_name] \\ \wedge u[building] = \text{"Watson"}))\}$$

Core Relational Algebra (Recap)



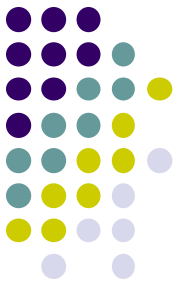
- **Union, intersection, and difference.** $R \cup S$ $R \cap S$
 - Usual set operations, both operands have the same relation schema.
- **Selection:** picking certain rows. $\sigma_c(R)$
- **Projection:** picking certain columns. $\Pi_{B,D,\dots}(R)$
- **Products** and **joins:** compositions of relations. $R \times S$ $R \bowtie_c S$
- **Grouping** and **Aggregation:** Grouping matching tuples $\gamma_{g1,g2,\dots,gm, F1(A1), \dots, Fn(An)}(R)$
- **Duplicate Elimination:** eliminates identical copies except one $\delta(R)$
- **Sorting:** Orders tuples based on a given criteria $\tau_x(R)$



Your Exercise (From CS3431)

- Remember the SQL Operators
- Know the semantics and Symbol of each one
- Given a SQL query in text, translate it to an algebraic expression
- What are the output schemas from each operator

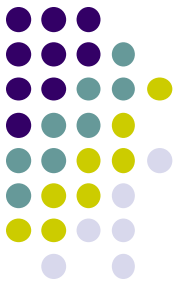
Algebra Behind the Query Language



Relational Algebra

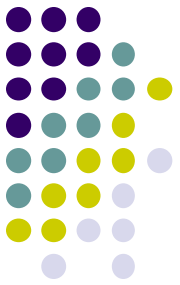
- Set of operators that operate on relations
- Operator semantics based on **Set** or **Bag** theory
- Relational algebra form underlying basis (and optimization rules) for SQL

```
SELECT pNumber, count(*) AS CNT  
FROM Student  
WHERE sNumber > 1  
GROUP BY pNumber;
```



Relational Algebra

- **Basic operators**
 - Set Operations (Union: $\cup$, Intersection: $\cap$, difference: $-$)
 - Select: σ
 - Project: π
 - Cartesian product: $\times$
 - rename: ρ
- More advanced operators, e.g., grouping and joins
- The operators take **one or two relations** as inputs and produce a **new relation** as an output
- One input $\rightarrow$ unary operator, two inputs $\rightarrow$ binary operator



Union over sets: $\cup$

- Consider two relations R and S that are ***union-compatible*** (same schema)

 R

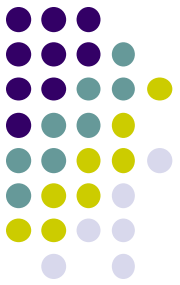
A	B
1	2
3	4

 S

A	B
1	2
3	4
5	6

 $R \cup S$

A	B
1	2
3	4
5	6



Binary Op.

Difference over sets: –

- $R - S$ are the tuples that appear in R and not in S
- R & S must be union-compatible
- Defined as:
 - $R - S = \{t \mid t \in R \text{ and } t \notin S\}$

$R - S \neq S - R$

R

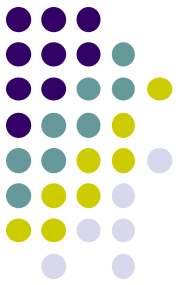
A	B
1	2
3	4

S

A	B
1	2
5	6

$R - S$

A	B
3	4



Intersection over sets: $\cap$

- Consider two Relations R and S that are union-compatible

R

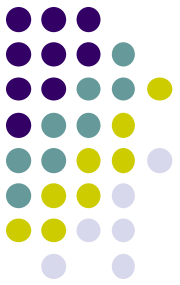
A	B
1	2
3	4

S

A	B
1	2
3	4
5	6

$R \cap S$

A	B
1	2
3	4



Selection: σ

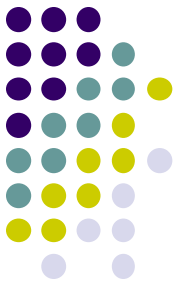
- **Select: $\sigma_c(R)$:**
 - c is a condition on R 's attributes
 - Select subset of tuples from R that satisfy **selection condition c**

R

A	B	C
1	2	5
3	4	6
1	2	7

$\sigma_{(C \geq 6)}(R)$

A	B	C
3	4	6
1	2	7



Selection: Example

R

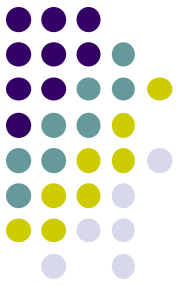
A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

$\sigma_{((A=B) \wedge (D>5))} (R)$

A	B	C	D
α	α	1	7
β	β	23	10

$\sigma_{(D>C)} (R)$

A	B	C	D
α	α	1	7
α	β	5	7



Project: π

- $\pi_{A1, A2, \dots, An}(R)$, with $A1, A2, \dots, An \in \text{attributes } A_R$
 - returns all tuples in R , but only columns $A1, A2, \dots, An$
- $A1, A2, \dots, An$ are called **Projection List**

R

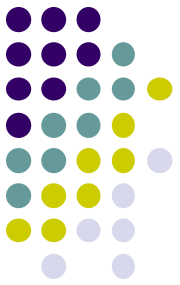
A	B	C
1	2	5
3	4	6
1	2	7
1	2	8

$\pi_{A, C}(R)$

A	C
1	5
3	6
1	7
1	8

Extended Projection: $\pi_L(R)$

Example



Rename column A to V

Compute this expression
and call it X

$\pi_{C, V \leftarrow A, X \leftarrow C*3+B}(R)$

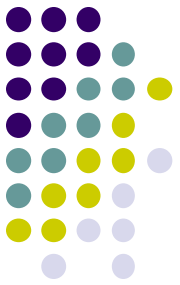
R

A	B	C
1	2	5
3	4	6
1	2	7
1	2	8

C	V	X
5	1	17
6	3	22
7	1	23
8	1	26

Cross Product (Cartesian Product): X

Binary Op.



- Each tuple in R joined with each tuple in S
- $R \times S = \{t \mid t \in R \text{ and } t \in S\}$

R

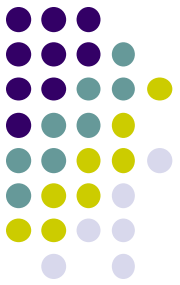
A	B
α	1
β	2

S

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

R X S

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b



Natural Join: $R \bowtie S$

- An implicit equality condition on the common columns

R

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

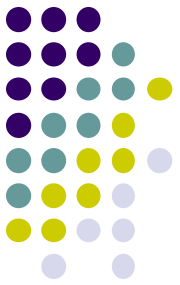
S

<i>B</i>	<i>D</i>	<i>E</i>
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

Implicit condition
($R.B = S.B$ and $R.D = S.D$)

$R \bowtie S$

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ



Theta Join: $R \bowtie_C S$

- A join based on any arbitrary condition C
- It is defined as :

$$R \bowtie_C S = (\sigma_C (R \times S))$$

R

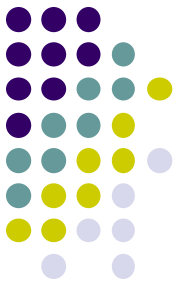
S

A	B
1	2
3	2

D	C
2	3
4	5
4	5

R $\bowtie_{R.A \geq S.C}$ ***S***

A	B	D	C
3	2	2	3



Duplicate Elimination: $\delta(R)$

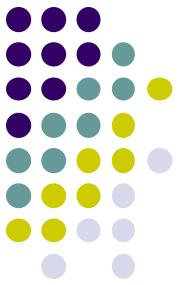
- Delete all duplicate records
- Convert a **Bag** (allows duplicates) to a **Set** (does not allow duplicates)

R

A	B
1	2
3	4
1	2
1	2

$\delta(R)$

A	B
1	2
3	4



Grouping & Aggregation operator: γ

- **Grouping & Aggregate operation** in relational algebra

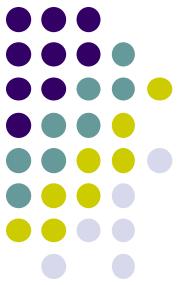
$\gamma_{g1, g2, \dots, gm, F1(A1), F2(A2), \dots, Fn(An)}(R)$

Group by these columns
(can be empty)

Aggregation functions applied
over each group

avg: average value
min: minimum value
max: maximum value
sum: sum of values
count: number of values

Grouping & Aggregation Operator: Example



R

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

S

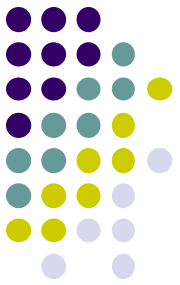
<i>branch_name</i>	<i>account_number</i>	<i>balance</i>
Perryridge	A-102	400
Perryridge	A-201	900
Brighton	A-217	750
Brighton	A-215	750
Redwood	A-222	700

$\gamma_{\text{sum}(c)}(R)$

sum(c)
27

$\gamma_{\text{branch_name}, \text{sum}(\text{balance})}(S)$

<i>branch_name</i>	sum(balance)
Perryridge	1300
Brighton	1500
Redwood	700



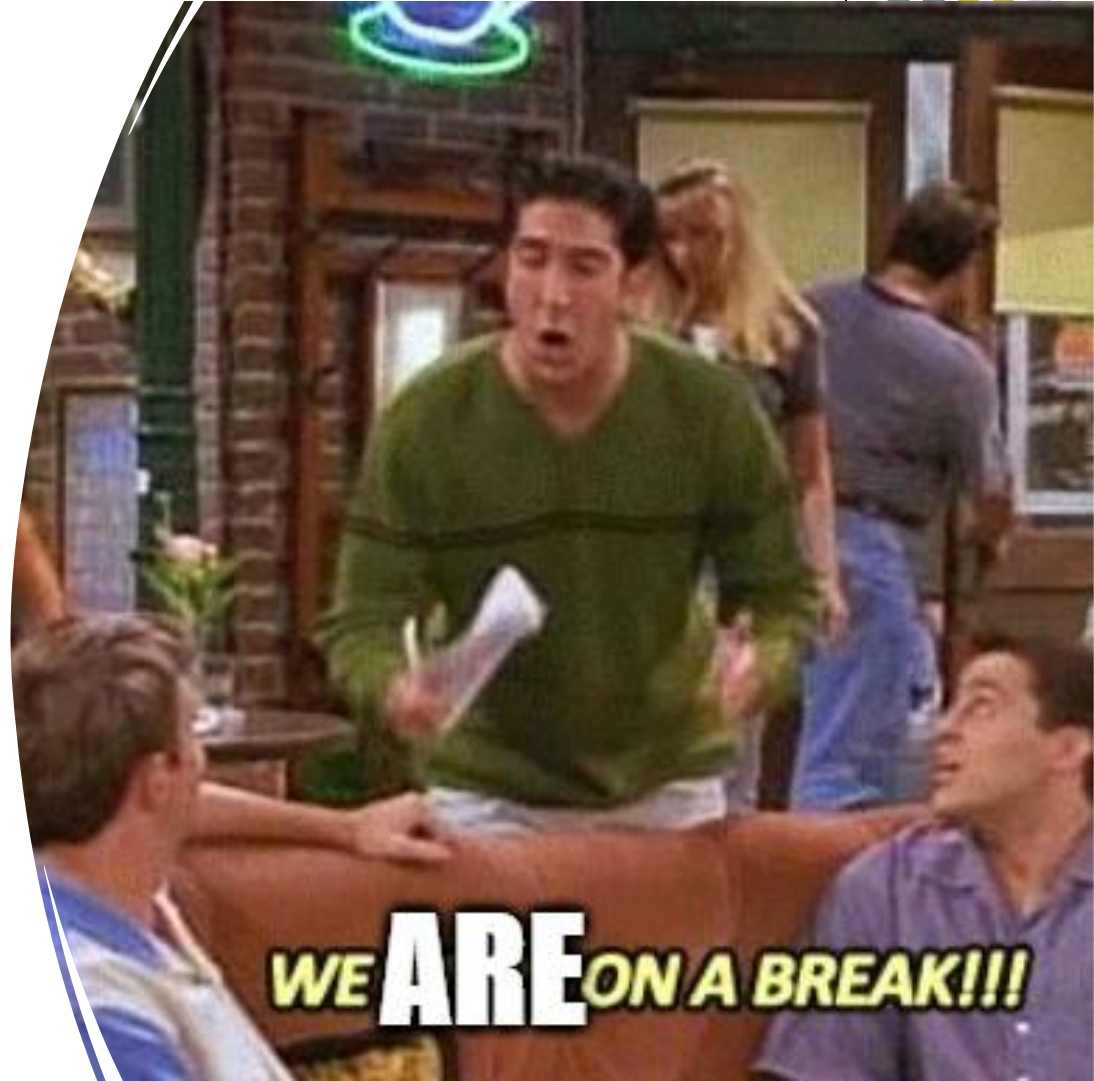
Assignment Operator: $\leftarrow$

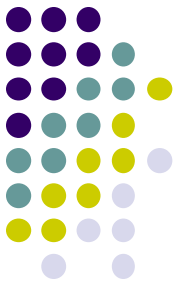
- **Write query as a sequence of lines consisting of:**
 - Series of assignments
 - Result expression containing the final answer
- May use a variable multiple times in subsequent expressions
- **Example:**
 - $R1 \leftarrow (\sigma_{((A=B) \wedge (D>5))} (R - S)) \cap W$
 - $R2 \leftarrow R1 \bowtie_{(R.A = T.C)} T$
 - $\text{Result} \leftarrow R1 \cup R2$



Break

- Now we will take a few minutes break
- Make sure we have everything you need for the **Quiz**



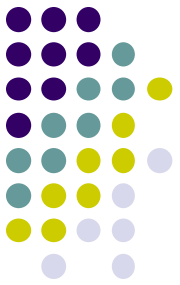


Banking Example

- *branch* (*branch_name*, *branch_city*, *assets*)
- *customer* (*customer_name*, *customer_street*, *customer_city*)
- *account* (*account_number*, *branch_name*, *balance*)
- *loan* (*loan_number*, *branch_name*, *amount*)
- *depositor* (*customer_name*, *account_number*)
- *borrower* (*customer_name*, *loan_number*)

For each query in the coming slides, in addition to the algebraic expression, try to write the SQL query

Example Queries



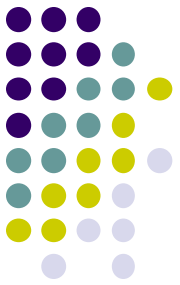
- *branch* (branch_name, branch_city, assets)
- *customer* (customer_name, customer_street, customer_city)
- *account* (account_number, branch_name, balance)
- *loan* (loan_number, branch_name, amount)
- *depositor* (customer_name, account_number)
- *borrower* (customer_name, loan_number)

Do it yourself 😊
(if time allows)

Find customer names having account balance below 100
or above 10,000

$\pi_{\text{customer_name}}(\text{depositor} \bowtie \pi_{\text{account_number}}(\sigma_{\text{balance} < 100 \text{ OR } \text{balance} > 10,000}(\text{account})))$

Example Queries



- *branch* (*branch_name*, *branch_city*, *assets*)
- *customer* (*customer_name*, *customer_street*, *customer_city*)
- *account* (*account_number*, *branch_name*, *balance*)
- *loan* (*loan_number*, *branch_name*, *amount*)
- *depositor* (*customer_name*, *account_number*)
- *borrower* (*customer_name*, *loan_number*)

Do it yourself 😊
(if time allows)

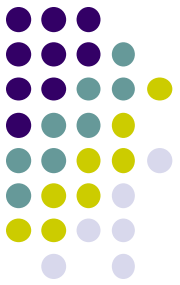
- Find all loans of over \$1200

$\sigma_{amount > 1200} (loan)$

- Find the loan number for each loan of an amount greater than \$1200

$\Pi_{loan_number} (\sigma_{amount > 1200} (loan))$

Example Queries (Cont'd)



Self Practice

- *branch* (*branch_name*, *branch_city*, *assets*)
- *customer* (*customer_name*, *customer_street*, *customer_city*)
- *account* (*account_number*, *branch_name*, *balance*)
- *loan* (*loan_number*, *branch_name*, *amount*)
- *depositor* (*customer_name*, *account_number*)
- *borrower* (*customer_name*, *loan_number*)

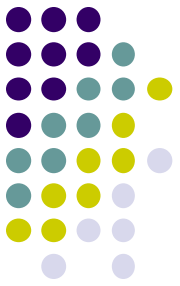
- Find the names of all customers who have a loan, an account, or both, from the bank

$$\Pi_{customer_name} (borrower) \cup \Pi_{customer_name} (depositor)$$

- Find the names of all customers who have a loan and an account at bank.

$$\Pi_{customer_name} (borrower) \cap \Pi_{customer_name} (depositor)$$

Example Queries



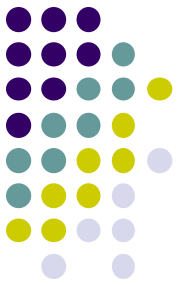
- *branch* (branch_name, branch_city, assets)
- *customer* (customer_name, customer_street, customer_city)
- *account* (account_number, branch_name, balance)
- *loan* (loan_number, branch_name, amount)
- *depositor* (customer_name, account_number)
- *borrower* (customer_name, loan_number)

Self Practice

Find customers' names who have neither accounts nor loans

$$\pi_{\text{customer_name}}(\text{customer}) - (\pi_{\text{customer_name}}(\text{borrower}) \cup \pi_{\text{customer_name}}(\text{depositor}))$$

Example Queries



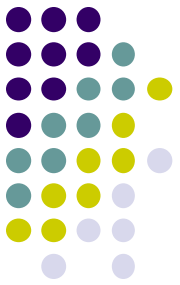
Self Practice

- *branch* (branch_name, branch_city, assets)
- *customer* (customer_name, customer_street, customer_city)
- *account* (account_number, branch_name, balance)
- *loan* (loan_number, branch_name, amount)
- *depositor* (customer_name, account_number)
- *borrower* (customer_name, loan_number)

For branches that gave loans > 100,000 or hold accounts with balances >50,000, report the branch name along whether it is reported because of a loan or an account

$$R1 \leftarrow \pi_{\text{branch_name}, \text{'Loan' As Type}} (\sigma_{\text{amount} > 100,000} (\text{loan}))$$
$$R2 \leftarrow \pi_{\text{branch_name}, \text{'Account' As Type}} (\sigma_{\text{balance} > 50,000} (\text{account}))$$
$$\text{Result} \leftarrow R1 \cup R2$$

Example Queries



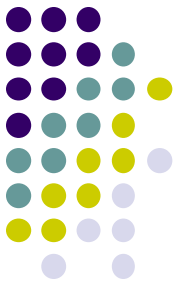
- *branch* (branch_name, branch_city, assets)
- *customer* (customer_name, customer_street, customer_city)
- *account* (account_number, branch_name, balance)
- *loan* (loan_number, branch_name, amount)
- *depositor* (customer_name, account_number)
- *borrower* (customer_name, loan_number)

Self Practice

Find customer names having loans with sum > 20,000

$\Pi_{\text{customer_name}} (\sigma_{\text{sum} > 20,000} (\gamma_{\text{customer_name}, \text{sum} \leftarrow \text{sum}(\text{amount})} (\text{loan} \bowtie \text{borrower})))$

Example Queries



Self Practice

- *branch* (branch_name, branch_city, assets)
- *customer* (customer_name, customer_street, customer_city)
- *account* (account_number, branch_name, balance)
- *loan* (loan_number, branch_name, amount)
- *depositor* (customer_name, account_number)
- *borrower* (customer_name, loan_number)

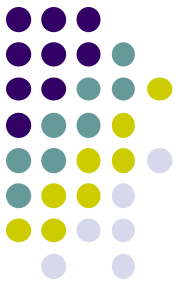
Find the branch name with the largest number of accounts

$R1 \leftarrow \gamma_{\text{branch_name}, \text{countAccounts} \leftarrow \text{count}(\text{account_number})}(\text{account})$

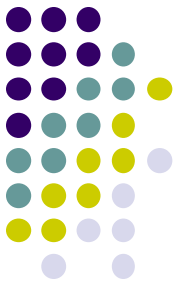
$R2 \leftarrow \gamma_{\text{Max} \leftarrow \text{max}(\text{countAccounts})}(R1)$

$\text{Result} \leftarrow \pi_{\text{branch_name}}(R1 \bowtie_{\text{countAccounts} = \text{Max}} R2)$

Summary of Relational-Algebra Operators



- Set operators
 - Union, Intersection, Difference
- Selection & Projection & Extended Projection
- Joins
 - Natural, Theta, Outer join
- Duplicate elimination
- Grouping & Aggregation



Quiz 4

- **Reminders:**
 - **Closed book**
 - **Individual work**
 - **You have 20 minutes**

